Lab Assignment 14.5

Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

• Display images in a grid layout.

• Use AI to:

o Optimize layout using CSS Grid.

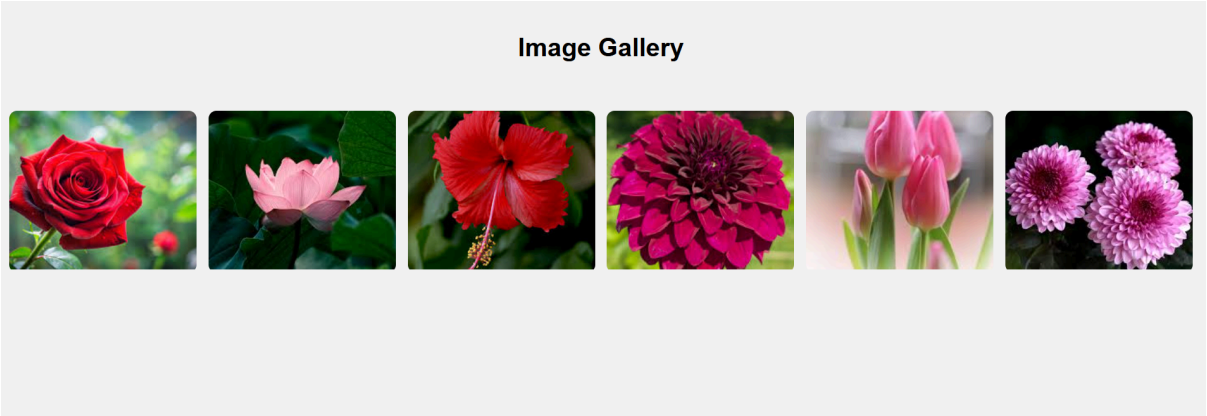
o Add hover zoom or overlay effects.

o Ensure mobile responsiveness.

```html
<!--Create an image gallery web page.
Display images in a grid layout.
Optimize layout using CSS Grid.
Add hover zoom or overlay effects.
Ensure mobile responsiveness.
-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f0f0f0;
        }

        h1 {
            text-align: center;
            padding: 20px;
        }

        .gallery {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
            gap: 15px;
            padding: 20px;
        }

        .gallery-item {
            position: relative;
            overflow: hidden;
            border-radius: 10px;
        }
```

```css
.gallery-item img {
    width: 100%;
    height: 200px;
    object-fit: cover;
    transition: transform 0.4s ease;
}

/* Hover Zoom */
.gallery-item:hover img {
    transform: scale(1.1);
}

/* Overlay Effect */
.overlay {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0,0,0,0.4);
    color: white;
    display: flex;
    justify-content: center;
    align-items: center;
    opacity: 0;
    transition: 0.3s;
    font-size: 18px;
}

.gallery-item:hover .overlay {
    opacity: 1;
}
```

```html
</head>
<body>
<h1>Image Gallery</h1>
<div class="gallery">
    <div class="gallery-item">
        <img src="Rose.jpg" alt="Rose">
        <div class="overlay">Rose</div>
    </div>
    <div class="gallery-item">
        <img src="Lotus.jpg" alt="Lotus">
        <div class="overlay">Lotus</div>
    </div>
    <div class="gallery-item">
        <img src="Hibiscus.jpg" alt="Hibiscus">
        <div class="overlay">Hibiscus</div>
    </div>
    <div class="gallery-item">
        <img src="dahlia.jpg" alt="Dahlia">
        <div class="overlay">Dahlia</div>
    </div>
    <div class="gallery-item">
        <img src="tulip.jpg" alt="Tulip">
        <div class="overlay">Tulip</div>
    </div>
    <div class="gallery-item">
        <img src="chrysanthemum.jpg" alt="Chrysanthemum">
        <div class="overlay">Chrysanthemum</div>
    </div>
</div>
</body>
</html>
```



Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

• Display user image, name, role, and social media links.

• Use AI to:

o Suggest modern UI colors and fonts.

o Design the card using Flexbox.

o Add hover animation and shadow effects

```html
<!--2.Create a profile card web page
Suggest modern UI colors and fonts.
Design the card using Flexbox.
Add hover animation and shadow effects-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Profile Card</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #e0f7fa;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            margin: 0;
        }

        .profile-card {
            background-color: #ffffff;
            border-radius: 15px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
            width: 300px;
            text-align: center;
            padding: 20px;
            transition: transform 0.3s ease, box-shadow 0.3s ease;
        }

        .profile-card:hover {
            transform: translateY(-10px);
            box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2);
        }

        .profile-image {
```

```css
.profile-image {
    width: 100px;
    height: 100px;
    border-radius: 50%;
    object-fit: cover;
    margin-bottom: 15px;
}

.profile-name {
    font-size: 1.5em;
    color: #00796b;
    margin-bottom: 5px;
}

.profile-title {
    font-size: 1em;
    color: #004d40;
    margin-bottom: 15px;
}

.profile-description {
    font-size: 0.9em;
    color: #004d40;
}
```

```html
<body>
    <div class="profile-card">
        <img src="https://via.placeholder.com/100" alt="Profile Image" class="profile-image">
        <div class="profile-name">Lily</div>
        <div class="profile-title">Web Developer</div>
        <div class="profile-description">Passionate about creating interactive and responsive web applications.</div>
    </div>
</body>
</html>
```

Task 3 -Develop a dynamic shopping cart system.

Requirements:

- Add, remove, and update product quantities.
- Display real-time total price.
- Use AI to:

o Manage cart state using JavaScript objects.

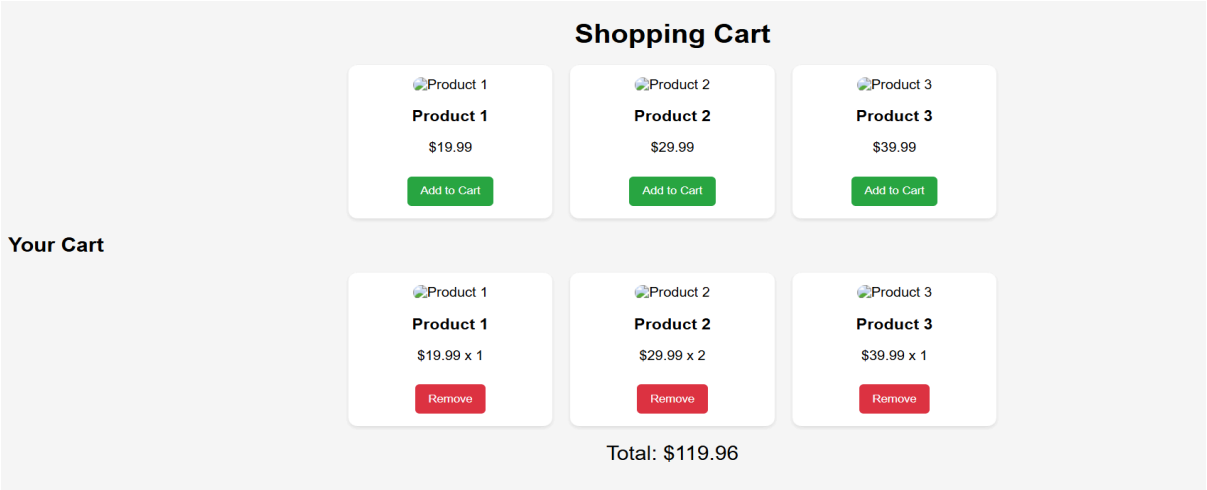
o Persist cart data in localStorage.

```html
<!--Develop a dynamic shopping cart system.
Add, remove, and update product quantities.
Display real-time total price.
Manage cart state using JavaScript objects.
Persist cart data in localStorage.
-->
<!Doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Shopping Cart</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 20px;
            background-color: #f9f9f9;
        }

        h1 {
            text-align: center;
            margin-bottom: 20px;
        }

        .product-list, .cart {
            display: flex;
            flex-wrap: wrap;
            gap: 20px;
            justify-content: center;
        }

        .product, .cart-item {
            background-color: white;
            border-radius: 10px;
            box-shadow: 0 2px 5px rgba(0,0,0,0.1);
            padding: 15px;
```

```
        width: 200px;
        text-align: center;
    }

    .product img, .cart-item img {
        width: 100%;
        height: 150px;
        object-fit: cover;
        border-radius: 10px;
    }

    .product button, .cart-item button {
        margin-top: 10px;
        padding: 10px 15px;
        background-color: #28a745;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    .cart-item button {
        background-color: #dc3545;
    }

    .total {
        text-align: center;
        margin-top: 20px;
        font-size: 1.5em;
    }
</style>
</head>
```

```
<body>
    <h1>Shopping Cart</h1>
    <div class="product-list">
        <!-- Products will be dynamically added here -->
    </div>

    <h2>Your Cart</h2>
    <div class="cart">
        <!-- Cart items will be dynamically added here -->
    </div>

    <div class="total">
        Total: $<span id="total-price">0.00</span>
    </div>

    <script>
        const products = [
            { id: 1, name: 'Product 1', price: 19.99, image: 'https://via.placeholder.com/200x150' },
            { id: 2, name: 'Product 2', price: 29.99, image: 'https://via.placeholder.com/200x150' },
            { id: 3, name: 'Product 3', price: 39.99, image: 'https://via.placeholder.com/200x150' },
        ];

        let cart = JSON.parse(localStorage.getItem('cart')) || {};

        function displayProducts() {
            const productList = document.querySelector('.product-list');
            productList.innerHTML = '';
            products.forEach(product => {
                const productDiv = document.createElement('div');
                productDiv.className = 'product';
                productDiv.innerHTML = `
                    <img src="${product.image}" alt="${product.name}">
                    <h3>${product.name}</h3>
                    <p>$${product.price.toFixed(2)}</p>
                    <button onclick="addToCart(${product.id})">Add to Cart</button>
                `;
```

```
    function updateCart() {
        const cartContainer = document.querySelector('.cart');
        cartContainer.innerHTML = '';
        let totalPrice = 0;

        Object.values(cart).forEach(item => {
            totalPrice += item.price * item.quantity;
            const cartItemDiv = document.createElement('div');
            cartItemDiv.className = 'cart-item';
            cartItemDiv.innerHTML = `
                <img src="${item.image}" alt="${item.name}">
                <h3>${item.name}</h3>
                <p>$${item.price.toFixed(2)} x ${item.quantity}</p>
                <button onclick="removeFromCart(${item.id})">Remove</button>
            `;
            cartContainer.appendChild(cartItemDiv);
        });
        document.getElementById('total-price').textContent = totalPrice.toFixed(2);
        localStorage.setItem('cart', JSON.stringify(cart));
    }
    displayProducts();
    updateCart();
  </script>
</body>
```



Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

• Admin, Editor, User roles.

• Conditional UI rendering.

• Use AI to:

o Design access control logic.

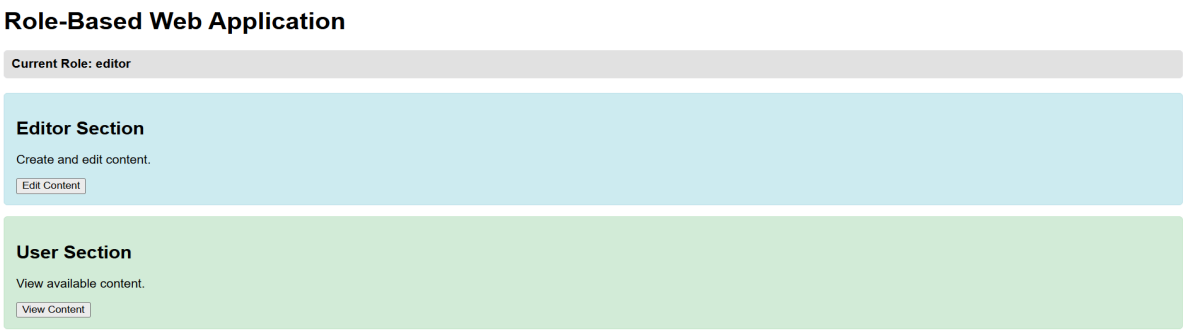
o Maintain secure state transitions.

o Create scalable UI components.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Role-Based Web Application</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .section {
            padding: 15px;
            margin: 15px 0;
            border-radius: 5px;
            display: none;
        }
        .admin {
            background-color: #f8d7da;
            border: 1px solid #f5c6cb;
        }
        .editor {
            background-color: #d1ecf1;
            border: 1px solid #bee5eb;
        }
        .user {
            background-color: #d4edda;
            border: 1px solid #c3e6cb;
        }
        .role-indicator {
            font-weight: bold;
            margin-bottom: 20px;
            padding: 10px;
            background-color: #e2e3e5;
            border-radius: 5px;
        }
    </style>
```

```html
</head>
<body>
    <h1>Role-Based Web Application</h1>
    <div class="role-indicator">Current Role: <span id="currentRole"></span></div>

    <div id="adminSection" class="section admin">
        <h2>Admin Section</h2>
        <p>Manage users and system settings.</p>
        <button onclick="performAction('admin')">Admin Action</button>
    </div>

    <div id="editorSection" class="section editor">
        <h2>Editor Section</h2>
        <p>Create and edit content.</p>
        <button onclick="performAction('editor')">Edit Content</button>
    </div>

    <div id="userSection" class="section user">
        <h2>User Section</h2>
        <p>View available content.</p>
        <button onclick="performAction('user')">View Content</button>
    </div>

    <script>
        const rolePermissions = {
            admin: ['adminSection', 'editorSection', 'userSection'],
            editor: ['editorSection', 'userSection'],
            user: ['userSection']
        };

        let currentUser = { role: 'editor' }; // Change to test different roles
```

```
        // Show permitted sections
        const permissions = rolePermissions[role] || [];
        permissions.forEach(sectionId => {
            const element = document.getElementById(sectionId);
            if (element) {
                element.style.display = 'block';
            }
        });

        document.getElementById('currentRole').textContent = role;
    }

    function performAction(action) {
        if (rolePermissions[currentUser.role].includes(action + 'Section')) {
            alert(`${currentUser.role} performed: ${action}`);
        } else {
            alert('Access Denied');
        }
    }

    function switchRole(newRole) {
        if (rolePermissions[newRole]) {
            currentUser.role = newRole;
            renderUIBasedOnRole(newRole);
        }
    }

    // Initialize
    renderUIBasedOnRole(currentUser.role);
</script>
</body>
</html>
```



Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

- Switch languages dynamically.
- Persist language preference.
- Use AI to:

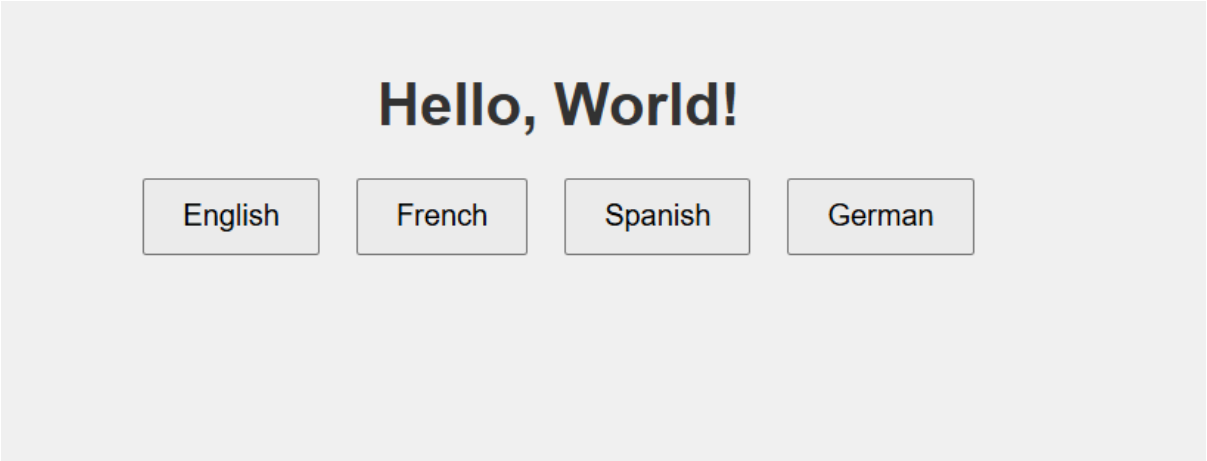
o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility.

```html
<!--Multi-Language Website
Design a multilingual website.
Switch languages dynamically.
Persist language preference.
Structure translation files.
Handle RTL/LTR layouts.
Improve accessibility.
Languages are English French spanish German-->
<!Doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Multilingual Website</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f0f0f0;
            margin: 0;
            padding: 20px;
        }

        h1 {
            text-align: center;
            color: #333;
        }

        .language-selector {
            display: flex;
            justify-content: center;
            margin-top: 20px;
        }

        .language-selector button {
            margin: 0 10px;
            padding: 10px 20px;
            font-size: 16px;
```

```html
        function setLanguage(lang) {
            document.getElementById('greeting').textContent = translations[lang];
            localStorage.setItem('preferredLanguage', lang);
        }

        // Load preferred language on page load
        window.onload = function() {
            const preferredLanguage = localStorage.getItem('preferredLanguage') || 'en';
            setLanguage(preferredLanguage);
        };
        // Handle RTL/LTR layouts
        function updateLayout(lang) {
            if (lang === 'ar' || lang === 'he') {
                document.body.style.direction = 'rtl';
            } else {
                document.body.style.direction = 'ltr';
            }
        }
        // Call updateLayout when language is changed
        function setLanguage(lang) {
            document.getElementById('greeting').textContent = translations[lang];
            localStorage.setItem('preferredLanguage', lang);
            updateLayout(lang);
        }
    </script>
</body>
</html>
```



Task 6 – Online Examination System

Create an online exam interface.

Requirements:

• Timer, questions, and submission.

• Prevent multiple submissions.

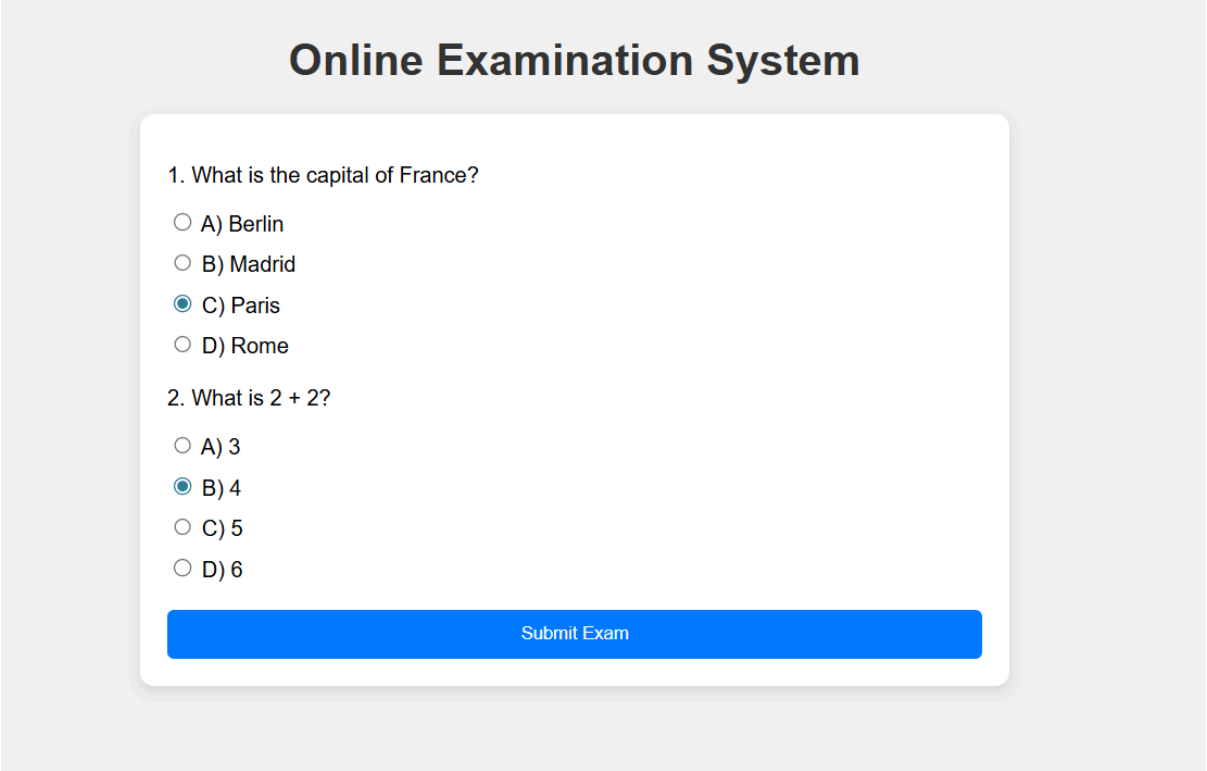
• Use AI to:

o Manage exam state.

o Improve exam UX.

o Ensure accessibility.

```html
<!--Task 6 – Online Examination System
Create an online exam interface.
Timer, questions, and submission.
Prevent multiple submissions.
Manage exam state.
Improve exam UX.
Ensure accessibility.
-->
<!Doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Examination System</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background-color: #f0f0f0;
            margin: 0;
            padding: 20px;
        }

        h1 {
            text-align: center;
            color: #333;
        }

        .exam-container {
            max-width: 600px;
            margin: 0 auto;
            background-color: #fff;
            padding: 20px;
            border-radius: 10px;
            box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
        }
```

```html
        .question {
            margin-bottom: 20px;
        }

        .options {
            list-style-type: none;
            padding: 0;
        }

        .options li {
            margin-bottom: 10px;
        }

        .submit-btn {
            display: block;
            width: 100%;
            padding: 10px;
            background-color: #007BFF;
            color: #fff;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }
    </style>
</head>
```

```html
</head>
<body>
    <h1>Online Examination System</h1>
    <div class="exam-container">
        <div class="question">
            <p>1. What is the capital of France?</p>
            <ul class="options">
                <li><input type="radio" name="q1" value="a"> A) Berlin</li>
                <li><input type="radio" name="q1" value="b"> B) Madrid</li>
                <li><input type="radio" name="q1" value="c"> C) Paris</li>
                <li><input type="radio" name="q1" value="d"> D) Rome</li>
            </ul>
        </div>
        <div class="question">
            <p>2. What is 2 + 2?</p>
            <ul class="options">
                <li><input type="radio" name="q2" value="a"> A) 3</li>
                <li><input type="radio" name="q2" value="b"> B) 4</li>
                <li><input type="radio" name="q2" value="c"> C) 5</li>
                <li><input type="radio" name="q2" value="d"> D) 6</li>
            </ul>
        </div>
        <button class="submit-btn">Submit Exam</button>
    </div>
</body>
</html>
```